

Attendify: AI-Powered Classroom Attendance System

1. Executive Summary

Attendify is an end-to-end intelligent classroom management solution designed to remove the friction of manual roll calls. By utilizing state-of-the-art computer vision models, specifically the InsightFace `buffalo_l` suite, Attendify can scan bustling, multi-person crowds and instantly cross-reference facial vectors against an enrolled student database. The system boasts an interactive React frontend, a robust Express/MongoDB backend, and an ultra-fast Python Flask Deep Learning engine.

2. System Architecture Overview

The platform leverages a microservices-inspired architecture splitting heavy GPU/CPU tensor computations from the lightweight web traversal pipelines.

2.1 Frontend (React + Vite + TailwindCSS)

The user interface is engineered for speed and clarity. It implements role-based dashboards utilizing `react-router-dom` for secure routing.

- **Zustand** is utilized for lightweight global state management (Authentication tokens).
- **Recharts** translates raw MongoDB aggregation pipelines into beautiful `AreaChart` and `BarChart` React nodes for the teacher's Analytical dashboards.
- **TailwindCSS** powers the responsive, dark-mode-first aesthetic, incorporating micro-animations and "Glassmorphic" layers for a premium user experience.

2.2 Backend (Node.js + Express + MongoDB)

The backend acts as the central nervous system, built on asynchronous ES Modules (ESM).

- **JWT Middleware:** Enforces stateless authentication, mapping users to 'student', 'teacher', or 'admin' roles.
- **RESTful Endpoints:** Provides isolated resource controllers for Users, Classes, and generic ML proxy tunneling.
- **MongoDB Aggregation:** The backend computes dynamic percentage rates, joining `AttendanceSession` metrics with `AttendanceRecord` historical data to feed the analytics UI instantly.

2.3 Machine Learning Engine (Python + ONNXRuntime)

The core Computer Vision logic is totally decoupled from Node.js, running on a highly-optimized Python Flask environment listening on port 5050.

- **InsightFace & SCRFD:** Detects multiple faces in under 150ms.
- **Dynamic Batching:** Once bounding boxes are detected, the system aligns all crops concurrently and feeds them as a *singular, chunked NumPy tensor array* to the ArcFace ResNet50 model.
- **Thread Capping:** ONNX runtime threads are explicitly capped (`intra_op_num_threads = 4`) to prevent CPU thrashing context switching, lowering a 60-person image inference from 17 seconds to a mere ~4.5 seconds.

3. The Technical Pipeline: Interactive Attendance

When a faculty member clicks "**Take Attendance**", a choreographed 4-step pipeline initiates.



Verification Interface

Step 1: Capture Phase

The React frontend requests access to the user's Webcam or triggers a File Upload mechanism. The captured blob is packaged into a `FormData` object and POSTed to the Node.js backend proxy.

Step 2: Processing & Proxied Tunnel

The Express proxy catches the image via `multer` into a temporary `uploads/temp` directory. The absolute filepath is relayed to the Python server via an HTTP proxy tunnel (`fetch` request to `http://localhost:5050/analyze`).

1. **Hi-Res Downscaling:** Python intercepts the image. If dimensions exceed 1080p equivalent, `cv2` automatically downscales the matrix. This prevents VRAM or RAM exhaustion.
2. **Cos-Sim Matrix Multiplication:** The 512-D float vectors generated for each detected face are multiplied linearly against the pre-loaded student database matrix (`known_embs.T`).
3. **Identity Allocation:** Utilizing `np.argmax`, the highest similarity match above a strict `0.35` Cosine Similarity threshold is permanently paired with a bounding box.

Step 3: GUI Verification & React Parsing

Express receives the Python JSON map and translates it to the React Component. React performs a set-intersection:

```
const detectedRolls = data.data.map(d => d.match_name.split(',')[1].trim());

setStudents(prev => prev.map(student => ({
  ...student,
  status: detectedRolls.includes(student.rollNumber) ? 'present' : 'absent'
})));
```

The teacher is presented with a heavily visual roster. Detected students are rendered green ([Lucide-React CheckCircle2](#)) while undetected drop to grey. Teachers have **Override Privileges** allowing manual clicks to toggle the derived AI state (to account for extreme lighting occlusion).

Step 4: Final Submission

Upon clicking "Submit Final Attendance", an [AttendanceSession](#) object is written to MongoDB. The bulk payload of manual statuses is converted directly into multiple [AttendanceRecord](#) elements in an atomic [.insertMany\(\)](#) transaction.

4. Analytics and Dynamic Visualization

Once Attendance is submitted, Recharts immediately absorbs the updated database matrix.



The MongoDB Aggregation Engine

To avoid [N+1](#) query flaws on the Node.js thread, we push metric calculation straight to the database layer.

- [\\$match](#): Identifies all [AttendanceSessions](#) belonging to the targeted Class.
- [\\$lookup](#): Equi-Joins the corresponding [AttendanceRecords](#).
- [\\$group](#): Mathematical accumulators evaluate overall class turnout ratios.

Students simultaneously render Radial Progress rings dynamically linked to their unique attendance ratio, enabling teachers to quickly isolate students falling behind the mandated 75% attendance rule.

5. Security Protocols

- **Encryption**: Passwords utilize Hash-Salts via [Bcrypt.js](#).

- **Stateless Tokens:** JWTs (JSON Web Tokens) are attached to user HTTP Request Headers, averting cross-site scripting cookie hijacking.
 - **Directory Isolation:** Model neural networks and user embeddings reside in an abstracted layer ([ml/buffalo_1/](#)) heavily git-ignored to prevent weight extraction from public repositories.
-

6. Closing Statement

Attendify seamlessly binds cutting-edge Computer Vision inference models with traditional web paradigms. The separation of concerns between standard Node/Express architectural layers and optimized generic-tensor-calculus Python boundaries ensures limitless vertical scalability. From sub-5-second multi-face detection to gorgeous analytical metric readouts, the platform successfully solves the modern pedagogical friction point of manual data entry.